Lecture 3: Evaluation

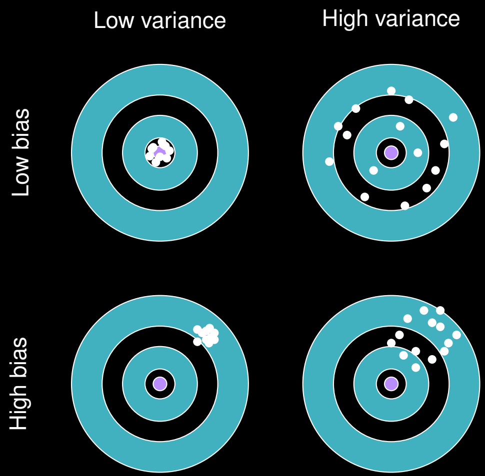
BIAS-VARIANCE TRADEOFF

Bias

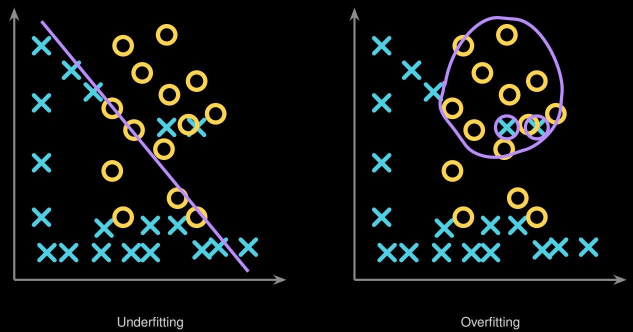
Error from an **overly simple** model. High bias misses the real pattern even on the training set: it **underfits**.

Variance

Error from **sensitivity to the training set**. High variance fits the noise in one sample and does not generalize: it **overfits**.



Overfitting and underfitting



Underfitting

A boundary too simple for the data: it misses points on both sides. High **bias**.

Overfitting

A boundary that hugs every training point, noise included. High **variance**.

Good fit

A boundary that follows the pattern and ignores the noise: the **sweet spot**.

Ways to mitigate overfitting and underfitting

If the model overfits

- **More data**, or **augmentation** to synthesize more.
- **Regularization**: weight decay, dropout.
- **Early stopping** at the validation minimum.
- **Reduce capacity**: fewer layers or parameters.
- Start from **pretrained weights** instead of scratch.

If the model underfits

- **Train longer**, or raise the learning rate.
- **Add capacity**: more layers or parameters.
- **Add features**, or a richer input representation.
- **Reduce regularization**: less weight decay or dropout.
- Check the loss and labels: a bug can look like underfitting.

Terminology

Ground truth \ Prediction	0	1
0	**TN** m_{00}	**FP** m_{01}
1	**FN** m_{10}	**TP** m_{11}

↑ missed disease

Be careful: some packages transpose this layout.

- **True positive** (m_{11}): an example of class 1 predicted as class 1.
- **False positive** (m_{01}): class 0 predicted as class 1. **Type I error.**
- **True negative** (m_{00}): class 0 predicted as class 0.
- **False negative** (m_{10}): class 1 predicted as class 0. **Type II error.**

Total instances $m = m_{00} + m_{01} + m_{10} + m_{11}$, and the **error rate** is $(m_{01} + m_{10})/m$.

Error rate hides imbalance. If 10% of examples are class 1 and 90% are class 0, calling *everything* class 0 already scores a 10% error rate. On a disease with 5% prevalence, always predicting "healthy" reaches **95% accuracy** and catches **zero** patients: every error it makes is an m_{10}, the one cell that matters clinically.

adapted from J. Pineau, COMP-551 (McGill)

Common measures

What machine learning calls them

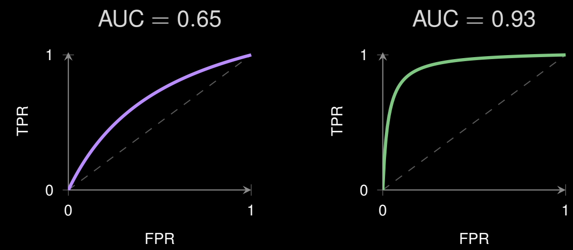
- $\text{Accuracy} = \dfrac{TP + TN}{TP + FP + FN + TN}$
- $\text{Precision} = \dfrac{TP}{TP + FP}$ of the declared positives
- $\text{Recall} = \dfrac{TP}{TP + FN}$ of the actual positives
- $F_1 = 2 \cdot \dfrac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$

What medicine calls them

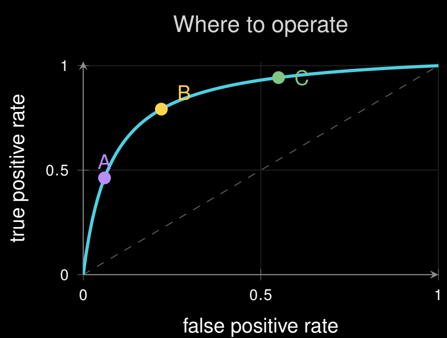
- $\text{Sensitivity} = \dfrac{TP}{TP + FN}$ the same as recall
- $\text{Specificity} = \dfrac{TN}{FP + TN}$ of the actual negatives
- $\text{False positive rate} = \dfrac{FP}{FP + TN}$
- $\text{PPV} = \text{precision}, \quad \text{NPV} = \dfrac{TN}{TN + FN}$

adapted from J. Pineau, COMP-551 (McGill)

Classification: ROC and AUC



A single number for the **whole curve**: higher AUC means better separation between classes.



ROC curves

Sweep the threshold from 1 to 0 and plot sensitivity against 1−specificity: threshold-free, good for comparing models.

Best-performing thresholds

AUC does not deploy. You must pick a point: **A** for high specificity (few false alarms), **C** for high sensitivity (screening), **B** as a balanced compromise.

adapted from Google, *Classification: ROC and AUC*